

Day 2 - HTTP, Express.js & REST API

Mục tiêu: Hiểu HTTP protocol và xây dựng REST API với Express.js

Sau Day 2, thực tập sinh có thể:

- Giải thích HTTP request/response cycle, các method và status code phổ biến
- Khởi tạo Express app, định nghĩa route với các HTTP method
- Viết middleware: built-in, custom, và error middleware
- Đọc dữ liệu từ `params`, `query`, `body`, `headers`
- Thiết kế và xây dựng REST API CRUD hoàn chỉnh với in-memory data
- Xử lý lỗi đúng chuẩn và trả về response JSON có cấu trúc nhất quán

Câu hỏi tìm hiểu trước

Trước khi học, tìm hiểu và trả lời các câu hỏi sau. Không cần đúng hoàn toàn — mục tiêu là có hình dung ban đầu trước khi đào sâu.

HTTP Protocol

- HTTP là gì? HTTPS khác HTTP ở điểm nào?
- HTTP request gồm những thành phần nào?
- HTTP response gồm những thành phần nào?
- HTTP/1.1, HTTP/2, HTTP/3 khác nhau ra sao?

HTTP Methods

- GET, POST, PUT, PATCH, DELETE dùng để làm gì?
- Idempotent nghĩa là gì? Method nào là idempotent?
- Safe method là gì? GET có phải safe method không?
- Khi nào dùng PUT, khi nào dùng PATCH?

HTTP Status Codes

- 2xx, 3xx, 4xx, 5xx đại diện cho loại gì?
- 200, 201, 204, 400, 401, 403, 404, 409, 500 nghĩa là gì?
- Tại sao phân biệt 401 và 403 quan trọng?

REST API

- REST là gì? RESTful API là gì?
- Stateless trong REST nghĩa là gì?
- Đặt tên resource trong URL như thế nào đúng chuẩn REST?
- JSON là gì, tại sao dùng JSON cho API?

Express.js

- Express.js là gì, vai trò trong Node.js ecosystem?
- Middleware là gì? Thứ tự khai báo middleware có quan trọng không?
- `app.use()` khác `app.get()` ở điểm nào?
- `next()` trong middleware dùng để làm gì?

Phần 0 - Chuẩn bị

Cài đặt dependencies cho Day 2:

```
# Trong thư mục project của bạn
npm install express
npm install --save-dev nodemon

# Thêm script vào package.json
# "dev": "nodemon src/index.js"
```

Cài Postman hoặc dùng Thunder Client (VS Code extension) để test API.

Kiểm tra:

```
node -e "require('express'); console.log('Express OK!')"
```

Phần 1 - HTTP Protocol

Request/Response Cycle

Khi browser (hoặc client) gửi request đến server:

Client	Server
--- HTTP Request ----->	
GET /api/users HTTP/1.1	
Host: localhost:3000	
Accept: application/json	
<-- HTTP Response -----	
HTTP/1.1 200 OK	
Content-Type: application/json	
{"users": [...]}	

Cấu trúc HTTP Request

Method	URL	Version
GET	/api/products?page=1	HTTP/1.1
Host: localhost:3000		
Authorization: Bearer <token>		
Content-Type: application/json		
{"name": "Laptop"} <- Body (chỉ có với POST/PUT/PATCH)		

Cấu trúc HTTP Response

Version	Status Code	Status Text
HTTP/1.1	200	OK
Content-Type: application/json		
Content-Length: 127		

```
{"id": 1, "name": "Laptop", "price": 25000000}
```

HTTP Methods

Method	Mục đích	Body	Idempotent	Safe
GET	Lấy dữ liệu	Không	Có	Có
POST	Tạo mới	Có	Không	Không
PUT	Thay thế hoàn toàn	Có	Có	Không
PATCH	Cập nhật một phần	Có	Không	Không
DELETE	Xóa	Không	Có	Không

HTTP Status Codes

1xx – Informational (hiếm dùng)

2xx – Success

200 OK – request thành công

201 Created – tạo mới thành công

204 No Content – thành công, không có body (vd: DELETE)

3xx – Redirection

301 Moved Permanently

302 Found (temporary redirect)

4xx – Client Error

400 Bad Request – request sai (thiếu field, sai format)

401 Unauthorized – chưa xác thực (chưa login)

403 Forbidden – đã xác thực nhưng không có quyền

404 Not Found – resource không tồn tại

409 Conflict – xung đột (vd: email đã tồn tại)

422 Unprocessable – validation failed

5xx – Server Error

500 Internal Server Error – lỗi server

502 Bad Gateway – server nhận response lỗi từ upstream

503 Service Unavailable – server quá tải hoặc bảo trì

Quy tắc thực tế:

- Đọc thành công → 200 OK
- Tạo thành công → 201 Created
- Xóa thành công → 204 No Content (hoặc 200 kèm message)
- Không tìm thấy → 404 Not Found
- Lỗi validation input → 400 Bad Request
- Chưa đăng nhập → 401 Unauthorized
- Không có quyền → 403 Forbidden
- Server crash → 500 Internal Server Error

Request data: 4 cách truyền thông tin

1. Route Params	/api/users/:id	-> req.params.id
2. Query String	/api/users?page=1&limit=10	-> req.query.page
3. Request Body	{"name": "Alice"}	-> req.body.name
4. Headers	Authorization: Bearer	-> req.headers.authorization

Phần 2 - Express.js Cơ bản

Setup Express App

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

// Middleware: parse JSON body
app.use(express.json());

// Middleware: parse URL-encoded body (form submit)
app.use(express.urlencoded({ extended: true }));

// Route đơn giản
app.get("/", (req, res) => {
  res.json({ message: "Hello, World!" });
});

// Khởi động server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Routing

```
// Basic routing - app.METHOD(path, handler)
app.get("/api/users", (req, res) => { /* ... */ });
app.post("/api/users", (req, res) => { /* ... */ });
app.put("/api/users/:id", (req, res) => { /* ... */ });
app.patch("/api/users/:id", (req, res) => { /* ... */ });
app.delete("/api/users/:id", (req, res) => { /* ... */ });

// Router - nhóm route liên quan
const router = express.Router();

router.get("/", getAllUsers);
router.post("/", createUser);
router.get("/:id", getUserById);
router.put("/:id", updateUser);
router.delete("/:id", deleteUser);
```

```
// Mount router
app.use("/api/users", router);
```

Request Object (req)

```
app.get("/api/products/:id", (req, res) => {
  // Route params: /api/products/42
  const { id } = req.params;      // "42" (string!)

  // Query string: /api/products?page=1&limit=10&sort=price
  const { page, limit, sort } = req.query;

  // Request body (POST/PUT/PATCH)
  const { name, price } = req.body;

  // Headers
  const token = req.headers.authorization;
  const contentType = req.headers["content-type"];

  // Method và URL
  console.log(req.method); // "GET"
  console.log(req.url);    // "/api/products/42"
  console.log(req.path);   // "/api/products/42"
});
```

Response Object (res)

```
app.get("/example", (req, res) => {
  // Gửi JSON
  res.json({ key: "value" });

  // Gửi với status code
  res.status(201).json({ id: 1, name: "Alice" });

  // Gửi status không có body
  res.status(204).send();

  // Redirect
  res.redirect("/new-url");

  // Set header trước khi gửi
  res.set("X-Custom-Header", "value");
  res.json({ data: "..." });
});
```

Middleware

Middleware là function nhận `(req, res, next)` và chạy theo thứ tự khai báo:

```
// Middleware signature
function myMiddleware(req, res, next) {
  // làm gì đó với req/res
  next(); // gọi next() để chuyển sang middleware/route tiếp theo
  // nếu không gọi next() -> request bị "treo"
}

// Global middleware - chạy cho tất cả route
app.use(express.json());
app.use(myMiddleware);

// Route-specific middleware
app.get("/protected", authenticate, (req, res) => {
  res.json({ data: "secret" });
});

// Middleware pipeline
app.use(logRequest);           // 1. Log
app.use(authenticate);         // 2. Auth
app.use("/api", apiRouter);    // 3. Route
app.use(errorHandler);         // 4. Error handler (4 params)
```

```
Request
  |
  v
logRequest middleware
  |
  v (next())
authenticate middleware
  |
  v (next())
Route handler
  |
  v (res.json())
Response
```

Custom Middleware ví dụ

```
// Logger middleware
function requestLogger(req, res, next) {
  const start = Date.now();
  console.log(`--> ${req.method} ${req.path}`);

  res.on("finish", () => {
    const duration = Date.now() - start;
    console.log(`<-- ${req.method} ${req.path} ${res.statusCode} ${duration}ms`);
  });

  next();
}
```

```

// Request ID middleware
function addRequestId(req, res, next) {
  req.id = `req_${Date.now()}_${Math.random().toString(36).slice(2, 9)}`;
  res.set("X-Request-ID", req.id);
  next();
}

// Validate JSON body middleware
function requireBody(fields) {
  return (req, res, next) => {
    const missing = fields.filter(f => req.body[f] === undefined);
    if (missing.length > 0) {
      return res.status(400).json({
        error: `Missing required fields: ${missing.join(", ")}`,
      });
    }
    next();
  };
}

// Dùng requireBody như factory middleware
app.post("/api/users", requireBody(["name", "email"]), createUser);

```

Error Handling Middleware

Express nhận biết error middleware qua **4 tham số** (err, req, res, next) :

```

// Trong route handler, dùng next(err) để chuyển lỗi xuống error middleware
app.get("/api/users/:id", async (req, res, next) => {
  try {
    const user = await getUserById(req.params.id);
    if (!user) {
      return res.status(404).json({ error: "User not found" });
    }
    res.json(user);
  } catch (err) {
    next(err); // chuyển lỗi xuống error middleware
  }
});

// Error middleware – PHẢI có 4 tham số, đặt CUỐI CÙNG
app.use((err, req, res, next) => {
  console.error(err.stack);

  // Nếu đã set statusCode trên error object
  const status = err.statusCode || 500;
  const message = err.message || "Internal Server Error";

  res.status(status).json({
    error: message,
  });
});

```

```

    // Chỉ show stack trong development
    ...(process.env.NODE_ENV === "development" && { stack: err.stack }),
  });
});

// 404 handler - khai báo trước error middleware, sau tất cả route
app.use((req, res) => {
  res.status(404).json({ error: `Cannot ${req.method} ${req.path}` });
});

```

Bài tập 1 - `exercises/01-http-basics.js`

Xem file `exercises/01-http-basics.js`

Bài tập 2 - `exercises/02-express-routing.js`

Xem file `exercises/02-express-routing.js`

Bài tập 3 - `exercises/03-middleware.js`

Xem file `exercises/03-middleware.js`

Phần 3 - REST API Design

RESTful Conventions

Resource: products

Đúng chuẩn REST:

GET	/api/products	- lấy danh sách
POST	/api/products	- tạo mới
GET	/api/products/:id	- lấy theo id
PUT	/api/products/:id	- thay thế hoàn toàn
PATCH	/api/products/:id	- cập nhật một phần
DELETE	/api/products/:id	- xóa

Nested resource:

GET	/api/products/:id/reviews	- lấy reviews của product
POST	/api/products/:id/reviews	- thêm review cho product

Sai - tránh dùng verb trong URL:

GET	/api/getProducts	✗
POST	/api/createProduct	✗
GET	/api/products/delete/:id	✗

Response Format nhất quán

Mỗi API nên trả về format đồng nhất để client dễ xử lý:

```

// Success response
{

```



```

    "success": true,
    "data": { ... },           // hoặc array
    "message": "Created successfully" // tùy chọn
  }

// List response (có pagination)
{
  "success": true,
  "data": [...],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 42,
    "totalPages": 5
  }
}

// Error response
{
  "success": false,
  "error": "Product not found",
  "code": "NOT_FOUND"         // tùy chọn, giúp client xử lý theo loại lỗi
}

```

CRUD API cho Products - Thiết kế

GET	/api/products	- Lấy tất cả (có filter theo category, sort theo price)
POST	/api/products	- Tạo mới (cần: name, price, category)
GET	/api/products/:id	- Lấy theo id
PUT	/api/products/:id	- Update toàn bộ fields
PATCH	/api/products/:id	- Update một số fields
DELETE	/api/products/:id	- Xóa

Bài tập 4 - `exercises/04-rest-api/`

Xem thư mục `exercises/04-rest-api/`

Phần 4 - Xây dựng REST API hoàn chỉnh

Kết hợp tất cả: routing, middleware, error handling, response format nhất quán.

Cấu trúc file đề xuất

```

exercises/04-rest-api/
├─ index.js           <- Entry point, khởi động server
├─ routes/
│   └─ products.js    <- Route handlers cho products
├─ middleware/
│   └─ index.js        <- Custom middleware

```

```
└─ data/
  └─ products.js  <- In-memory data store
```

index.js (entry point)

```
const express = require("express");
const app = express();
const productsRouter = require("./routes/products");
const { requestLogger, notFoundHandler, errorHandler } = require("./middleware");

const PORT = process.env.PORT || 3000;

// Global middleware
app.use(express.json());
app.use(requestLogger);

// Routes
app.get("/", (req, res) => {
  res.json({ message: "Products API v1.0", endpoints: ["/api/products"] });
});
app.use("/api/products", productsRouter);

// 404 và Error handler - PHẢI đặt cuối
app.use(notFoundHandler);
app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

module.exports = app;
```

data/products.js (in-memory store)

```
let products = [
  { id: 1, name: "Laptop", price: 25000000, category: "tech", inStock: true,
    createdAt: new Date() },
  { id: 2, name: "Phone", price: 15000000, category: "tech", inStock: false,
    createdAt: new Date() },
  { id: 3, name: "Desk", price: 5000000, category: "furniture", inStock: true,
    createdAt: new Date() },
];
let nextId = 4;

const getAll = (filter = {}) => {
  let result = [...products];
  if (filter.category) result = result.filter(p => p.category === filter.category);
  if (filter.inStock !== undefined) result = result.filter(p => p.inStock ===
    (filter.inStock === "true"));
  if (filter.sort === "price_asc") result.sort((a, b) => a.price - b.price);
}
```

```
    if (filter.sort === "price_desc") result.sort((a, b) => b.price - a.price);
    return result;
};

const getById = (id) => products.find(p => p.id === Number(id));

const create = (data) => {
    const product = { id: nextId++, ...data, createdAt: new Date() };
    products.push(product);
    return product;
};

const update = (id, data) => {
    const index = products.findIndex(p => p.id === Number(id));
    if (index === -1) return null;
    products[index] = { ...products[index], ...data, id: Number(id) };
    return products[index];
};

const remove = (id) => {
    const index = products.findIndex(p => p.id === Number(id));
    if (index === -1) return false;
    products.splice(index, 1);
    return true;
};

module.exports = { getAll, getById, create, update, remove };
```

Bài tập về nhà - homework/todo-api.js

Nâng cấp TODO app từ Day 1 thành REST API với Express.

Xem file `homework/todo-api.js`

Tiêu chí đánh giá

Tiêu chí	Mô tả	Điểm
Bài 1 - HTTP Basics	Trả lời đúng câu hỏi lý thuyết, demo curl	10
Bài 2 - Express Routing	Routes đúng, đọc params/query/body đúng	20
Bài 3 - Middleware	Middleware chạy đúng thứ tự, error handler đúng	20
Bài 4 - REST API	CRUD đầy đủ, status code đúng, response format nhất quán	25
Homework - TODO API	Đủ CRUD, có validation, error handling chuẩn	25

Pass Day 2: >= 70 điểm

Tài liệu tham khảo

Bắt buộc đọc:

- [Express.js - Getting started](#)
- [Express.js - Routing](#)
- [Express.js - Middleware](#)
- [Express.js - Error handling](#)
- [MDN - HTTP overview](#)
- [MDN - HTTP methods](#)
- [MDN - HTTP status codes](#)

Đọc thêm:

- [RESTful API Design - Best Practices](#)
- [HTTP Status Codes cheat sheet](#)
- [Express.js API Reference](#)

Tool:

- [Postman](#) - test API
- [Thunder Client](#) - VS Code extension

Stuck quá 15 phút với một bài -> hỏi mentor ngay.